

# Beta-v2 Email Investigation Peer Setup Guide



A simple first-time checklist for a teammate who pulled the code and received a copy of backend\env.

WINDOWS + POWERSHELL

NO REAL KEYS INCLUDED

LOCALHOST ONLY

## The simple answer

Yes - they still need local setup. Git sends the code, but it does not send the Python environment, login users, security keys, or audit files.

| STEP             | DO THEY DO IT?          | EASY REASON                                                             |
|------------------|-------------------------|-------------------------------------------------------------------------|
| 5. Security keys | YES - once per computer | Each computer should protect its own login sessions and audit chain.    |
| 6. Provider keys | CHECK ONLY              | If the received .env already has complete keys, do not type them again. |
| 7. Login account | YES - once              | Users are local and are not included in Git. There is no default login. |
| 8. Start the app | YES - every use         | This starts the backend on 8010 and the website on 3000.                |

### Security keys are not provider keys

Authentication and audit keys protect the local SOC login. LeakOSINT and SerpAPI keys allow external searches. Do not use one type of key in place of the other.

Setup path: pull code > create .env > save backend\env > generate local security keys > create user > run.py

# Prepare the computer

Run these commands in Windows PowerShell. Replace the example folder with the real repository location.

**1**

## Open the repository and pull the newest code

Why: This makes sure the teammate has the same Beta-v2 files as you.

```
Set-Location D:\path\to\public-osint
git pull
Set-Location .\Beta-v2
```

**2**

## Create the local Python environment

Why: The .venv folder is not pushed to Git and must be created on every computer. Python 3.11 or newer is required.

```
py -3.11 -m venv .venv
```

**3**

## Install the required packages

Why: This installs FastAPI, the email validator, and the other packages used by the current code. Run it again after a pull when requirements change.

```
.\.venv\Scripts\python.exe -m pip install `
-r .\backend\requirements.txt
```

**4**

## Save the received environment file in the exact folder

Why: Beta-v2 reads only Beta-v2\backend\.env. It does not read a root .env or the older backend configuration.

```
notepad .\backend\.env
```

### Check the filename carefully

Correct: Beta-v2\backend\.env

Wrong: Beta-v2\.env or Beta-v2\backend\.env.txt. In File Explorer, enable file-name extensions if needed.

### Do not overwrite a supplied .env

If the teammate already received a complete backend\.env, do not copy .env.example over it. The example contains empty placeholders.

# Security keys and provider check

This page answers exactly what to do when the complete provider configuration was already sent to the teammate.

## Step 5 - Generate unique local security keys

### Do this only on a fresh peer setup, before the first investigation

Open backend\.env and make only these two values blank: AUTH\_SESSION\_SECRET= and AUDIT\_HMAC\_KEY=. Do not delete the LeakOSINT or SerpAPI values.

```
notepad .\backend\.env

.\.venv\Scripts\python.exe `
.\backend\scripts\configure_security.py
```

Expected message: Configured distinct local authentication and audit secrets. The generated values are saved without being printed.

### Already used this installation? Do not rotate blindly

If an audit log already contains records, keep its audit key. The setup script will refuse an unsafe replacement. Ask the system administrator before changing an existing installation.

## Step 6 - Check the provider settings

If the received .env already contains complete values, there is nothing to enter again. Confirm only these lines:

```
AUTH_COOKIE_SECURE=false

EMAIL_INVESTIGATION_BREACH_ENABLED=true
EMAIL_INVESTIGATION_BREACH_API_KEY=<complete-valid-LeakOSINT-token>

EMAIL_INVESTIGATION_DORK_ENABLED=true
SERPAPI_KEY=<complete-valid-SerpAPI-key>
```

| SETTING                  | WHAT IT DOES                                                      |
|--------------------------|-------------------------------------------------------------------|
| LeakOSINT                | Provides exact-email breach metadata and allowed contact fields.  |
| SerpAPI                  | Provides bounded Google/Bing public-web discovery.                |
| AUTH_COOKIE_SECURE=false | Required for local HTTP at 127.0.0.1. Use true only behind HTTPS. |

### Important

Complete keys are required. A masked value such as MASKED\_KEY\*\*\*\*\* will not work. Other scraper keys do not replace the dedicated LeakOSINT and SerpAPI keys.

# Create the login, start, and investigate

The provider configuration alone is not enough. Every computer also needs a local SOC user.

## Step 7 - Create the local analyst account

```
.\.venv\Scripts\python.exe `
.\backend\scripts\create_soc_user.py `
--username analyst `
--role investigator `
--role breach_pii_viewer
```

Enter and confirm a password of at least 12 characters. Nothing appears while the password is typed. Both roles are required for the restricted contact-record option.

### If the user already exists

Run the same command with `--replace` at the end. The correct option is `--replace`, not `--repl`.

## Step 8 - Start both servers

```
.\.venv\Scripts\python.exe .\run.py
```

Wait for BOTH SERVERS STARTED SUCCESSFULLY!, then open <http://127.0.0.1:3000/>. Keep PowerShell open. Press Ctrl+C when finished.

## Run the first email investigation

- |                                                                            |                                                                     |
|----------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1 Log in with the local analyst account.                                   | 2 Click Email Investigation in the top navigation.                  |
| 3 Enter exactly one complete target email address.                         | 4 Enter the official case ID and choose the correct reason.         |
| 5 Tick the authorization confirmation box.                                 | 6 Keep Breach intelligence selected when breach checking is needed. |
| 7 Select Restricted breach contact records only when the case requires it. | 8 Keep public-web discovery selected when SerpAPI search is needed. |
| 9 Click Investigate Email and review every source and limitation.          | 10 If the page looks old after a pull, press Ctrl+F5.               |

### Restricted access is deliberate

The restricted checkbox appears only for a user with both roles. Every restricted access attempt and every returned field set is audited before values are shown.

# What the output should show

The dashboard can complete successfully even when a provider returns no matching record. Read the provider status before deciding that setup is broken.

## Expected result sections

- 1 Validation & Domain Intelligence: normalized email, provider category, DNS, MX records, and mail provider.
- 2 Breach Intelligence: LeakOSINT source metadata, risk summary, and exact-address findings.
- 3 Categorized Public-Web Discovery: bounded Google/Bing results from SerpAPI.
- 4 Domain Email Harvest: addresses derived only from returned search titles and snippets.
- 5 Gravatar / Associated Identity: public profile details if available.
- 6 Limitations & Analyst Handling: warnings that results are leads and require verification.

### When will name, phone, and address tiles appear?

Only when all four conditions are true: a complete valid LeakOSINT key is configured; the provider returns a row containing the exact target email; the analyst has both roles; and the restricted checkbox was selected. The system does not invent missing values.

| STATUS             | EASY MEANING                                                                                                            |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| FOUND              | The provider returned one or more matching records.                                                                     |
| NO RESULTS         | The search completed but found no exact match. This is not proof of no exposure.                                        |
| NOT CONFIGURED     | The required key is missing from this computer's backend\env.                                                           |
| DISABLED / SKIPPED | The feature or UI option is off, or its query limit is zero.                                                            |
| PROVIDER ERROR     | The key may be invalid or expired, quota may be exhausted, the provider may be down, or outbound access may be blocked. |
| PARTIAL            | Some checks succeeded and others failed. Review each section separately.                                                |

## Safe checks while the app is running

```
Invoke-RestMethod http://127.0.0.1:8010/ready
```

```
Invoke-RestMethod `
    http://127.0.0.1:8010/api/v1/investigation/diagnostics/keys
```

Readiness should return status: ready. Diagnostics shows only configured/active flags; it does not expose keys or prove that a key is valid or has quota.

### Fast troubleshooting

Login 401: check the local user/password. Login or launch 503: run security setup and create an active user. 403 insufficient role: recreate the user with both roles. CSRF failure: sign out, press Ctrl+F5, and sign in again. 422: complete the email, case, reason, and authorization fields. Port 3000/8010 in use: stop the earlier run.py window and retry.

### Protect credentials and evidence

Never push backend\env or backend\runtime. Use an approved secure secret-sharing method in future, remove unnecessary chat copies after receipt, and rotate provider keys if exposure is suspected. Passwords, hashes, tokens, payment data, government IDs, medical data, DOB, IP, and device identifiers remain suppressed and are excluded from exports.